Objectives

- **Application Development:** Create ShopSphere, a complete product management application.
- **Architecture Adoption:** Use the 3-Layer Architecture for clean code organization.
- **Module Separation:** Code is divided into distinct modules: **Presentation, Business Logic, and Persistence**.
- **Core Functionality:** Implement the four basic functions: Create, Read, Update, and Delete (CRUD).
- **Dependency Rule:** Ensure the standard call flow is followed: `Controller` -> `Service` -> `Repository`.

System Architecture (Theory & Structure)

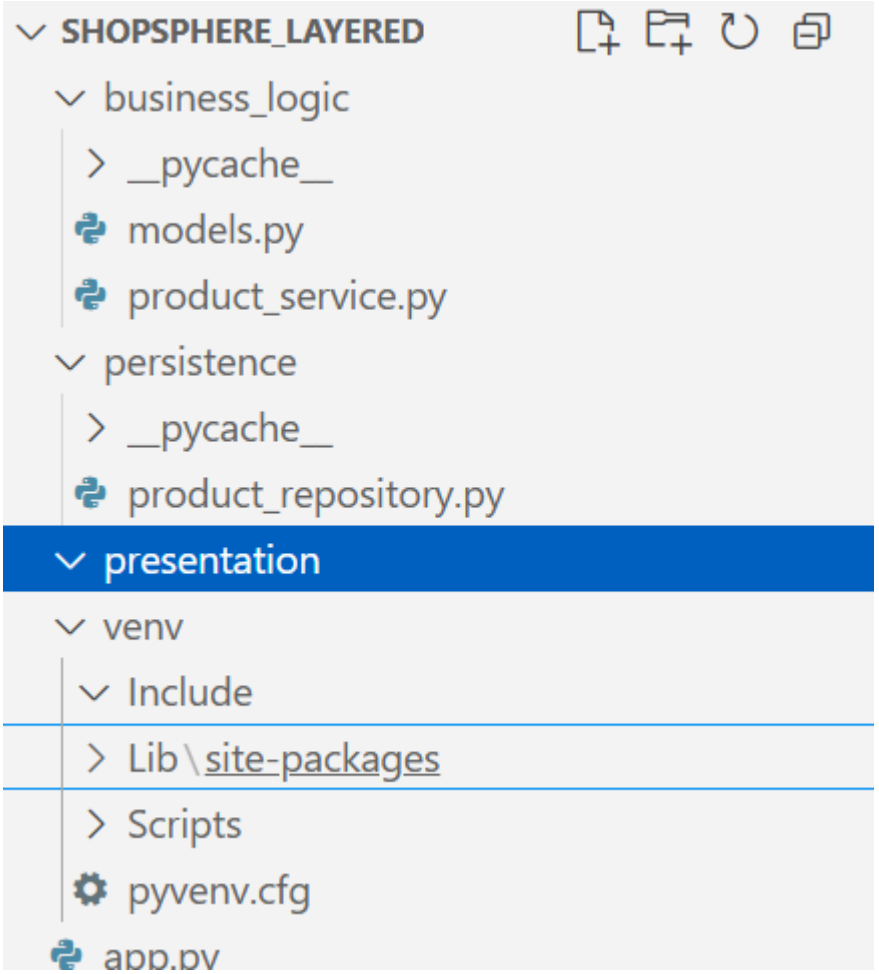
The system is divided into 3 layers with specific roles:

- **Presentation Layer (Controller/API):**
  - **Responsibility:** Handle HTTP requests from the user.
  - **Action:** Call the Service layer and return the result (JSON).
- **Business Logic Layer (Service):**
  - **Responsibility:** Contain business rules (e.g., checking if price > 0).
  - **Action:** Verify logic before allowing data to be saved.
- **Persistence Layer (Repository):**
  - **Responsibility:** Directly interact with the data storage.
  - **Action:** In this lab, a Python `Dictionary` in temporary memory is used to simulate a database.

Data Flow Diagram:

`Request` -> `Controller` -> `Service` -> `Repository` -> `Data Store`

Project Structure



Detailed Implementation

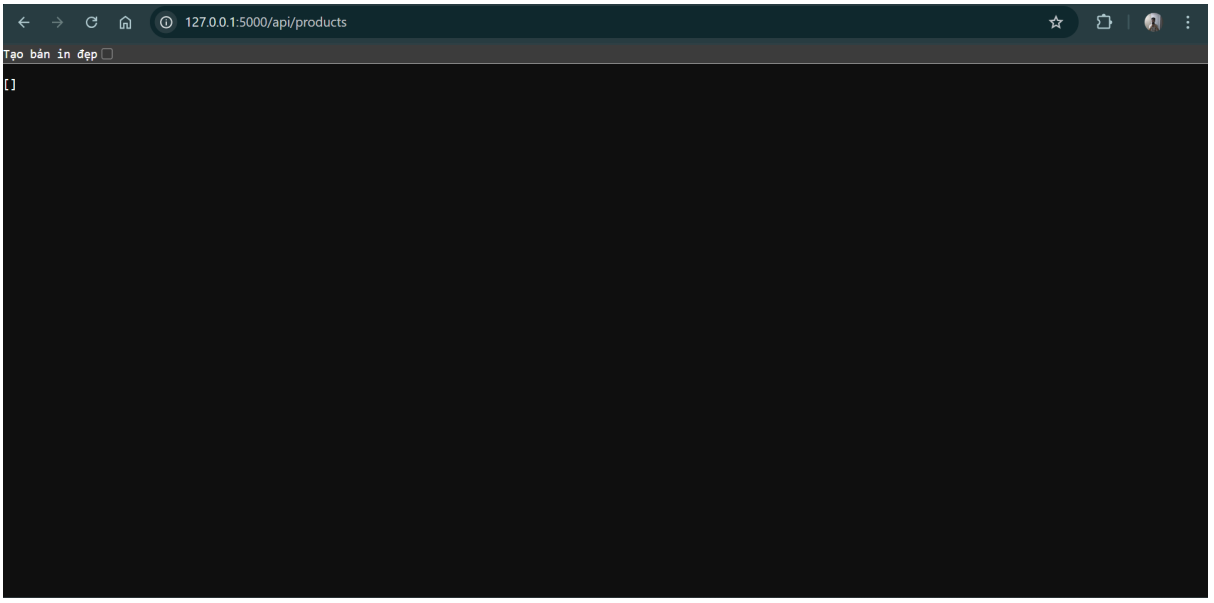
- **Persistence Layer (Repository):** Uses the `product_db` Dictionary as a temporary database.
- **Business Logic Layer (Service):** Notable for checking data validity (e.g., the `create_product` function includes a price check).
- **Presentation Layer (Controller):** Uses the Flask library to define Endpoints for `PUT` (Update) and `DELETE` operations.

Testing Results

- **Scenario 1: Server Startup:** Server successfully runs on port 5000.

```
(venv) PS C:\Users\truon\shopsphere_layered> python app.py
 * Serving Flask app 'app'
 * Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on http://127.0.0.1:5000
Press CTRL+C to quit
 * Restarting with stat
 * Debugger is active!
 * Debugger PIN: 946-662-458
127.0.0.1 - - [15/Jan/2026 14:02:45] "GET / HTTP/1.1" 404 -
127.0.0.1 - - [15/Jan/2026 14:02:50] "GET /api/products HTTP/1.1" 200 -
```

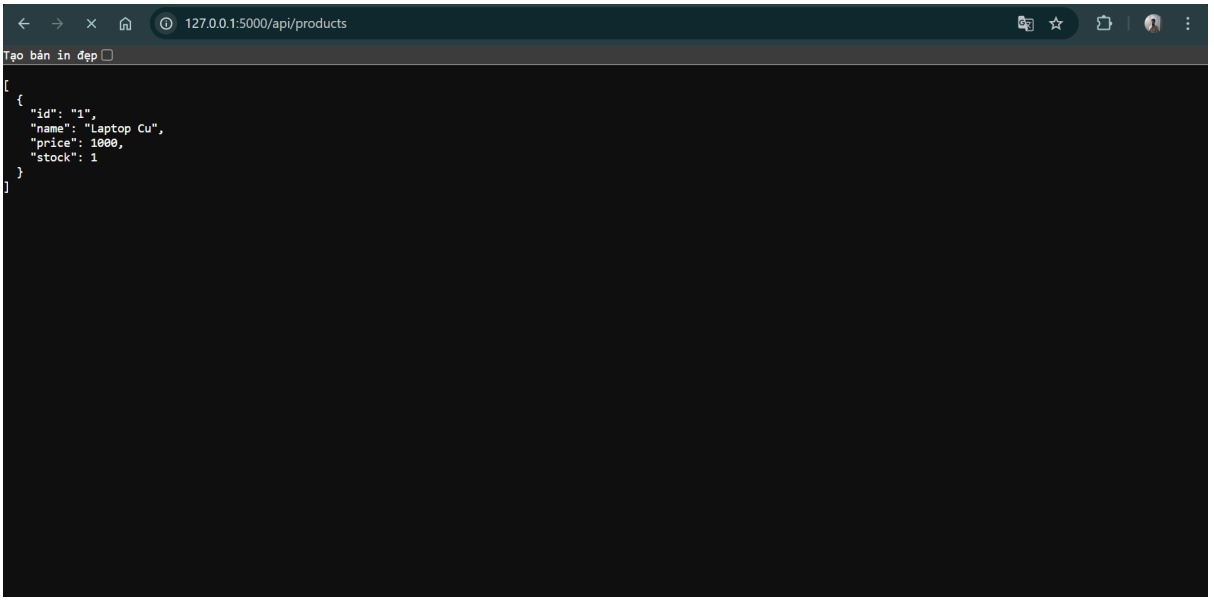
Scenario 2: Get Empty List (GET): The API returns [] when there is no data.



Scenario 3: Add New Product (POST): Add the product "Laptop Dell".

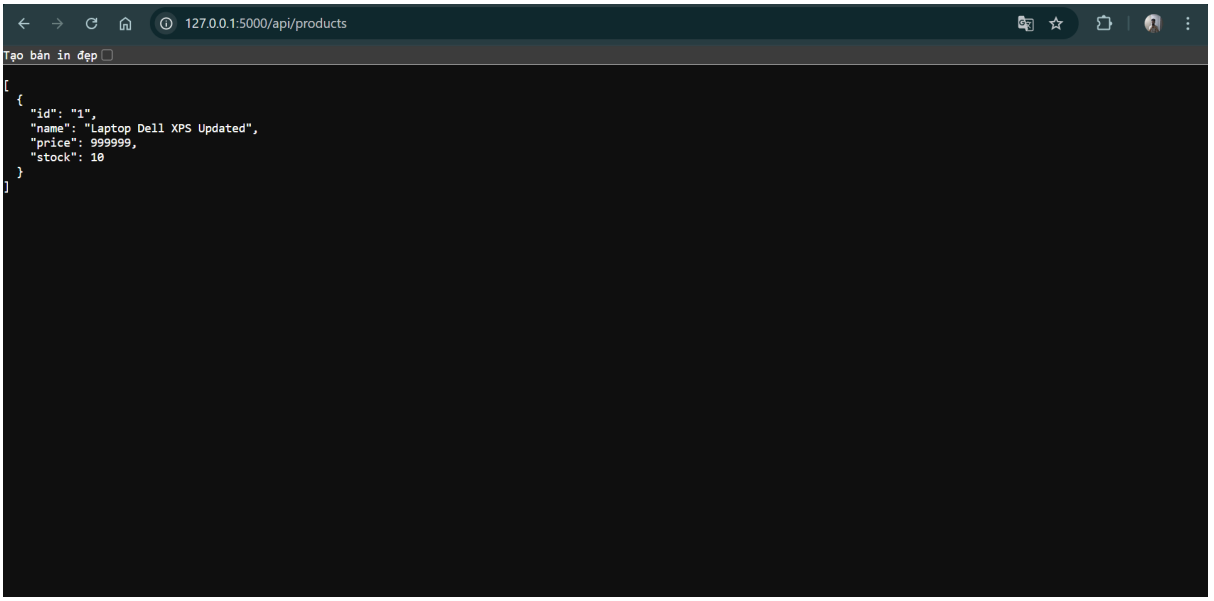
```
(venv) PS C:\Users\truon\shopsphere_layered> Invoke-RestMethod -Method Post -Uri "http://127.0.0.1:5000/api
/products" -ContentType "application/json" -Body '{"name": "Laptop Cu", "price": 1000, "stock": 1}'

id name      price stock
-- ----      ----- -----
1  Laptop Cu  1000     1
```

- 

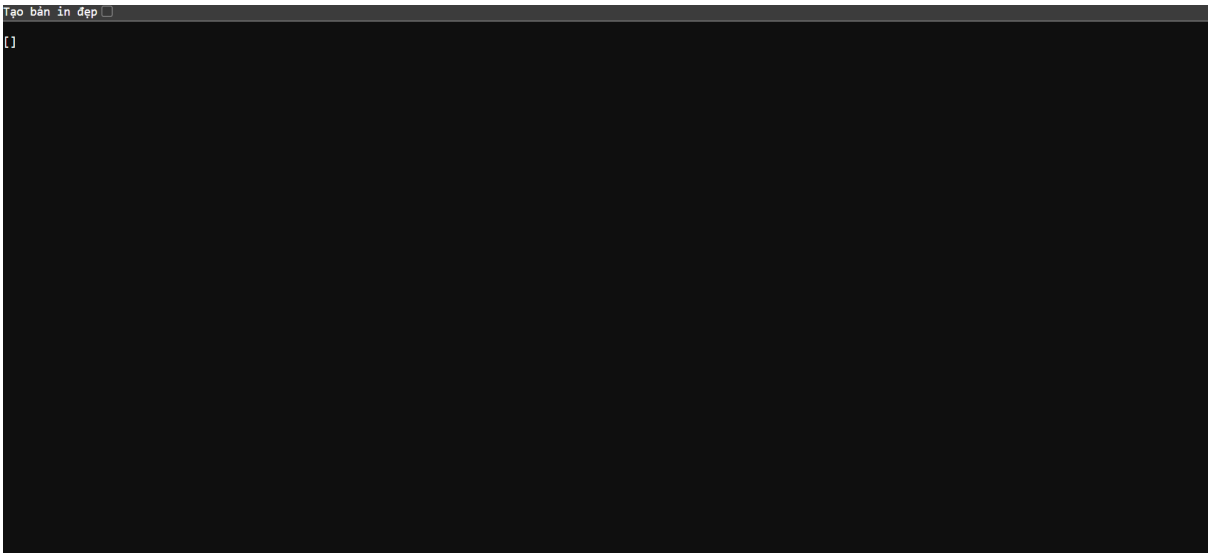
Scenario 4: Update Product (PUT): Update the price for product ID 1.

- ```
  (venv) PS C:\Users\truon\shopsphere_layered> Invoke-RestMethod -Method Put -Uri "http://127.0.0.1:5000/api/products/1" -ContentType "application/json" -Body '{"name": "Laptop Dell XPS Updated", "price": 999999, "stock": 10}'

  id name                    price stock
  -- ----                    ----- -----
  1  Laptop Dell XPS Updated 999999    10
  ```
- 

Scenario 5: Delete Product (DELETE): Remove product ID 1 from the system.

- ```
  (venv) PS C:\Users\truon\shopsphere_layered> Invoke-RestMethod -Method Delete -Uri "http://127.0.0.1:5000/api/products/1"

  message
  -------
  Đã xóa sản phẩm thành công
  ```
- 

Challenges & Solutions

Issue	Cause	Solution
Error `ModuleNotFoundError: No module named 'flask'`	Flask is not installed, or VS Code is incorrectly detecting the virtual environment (venv).	Run `pip install flask` in the venv and re-select the Python Interpreter in VS Code.
Data lost upon code modification	Using a `dict` variable in RAM; Flask server reload resets the RAM.	Accept the lab's limitation, or write a script to automatically create sample data upon `app.py` startup.
Update error: ID not found	Server reload caused the old ID to be lost.	Always check the list (GET) first to retrieve the correct ID before performing an Update.